

Collection framework

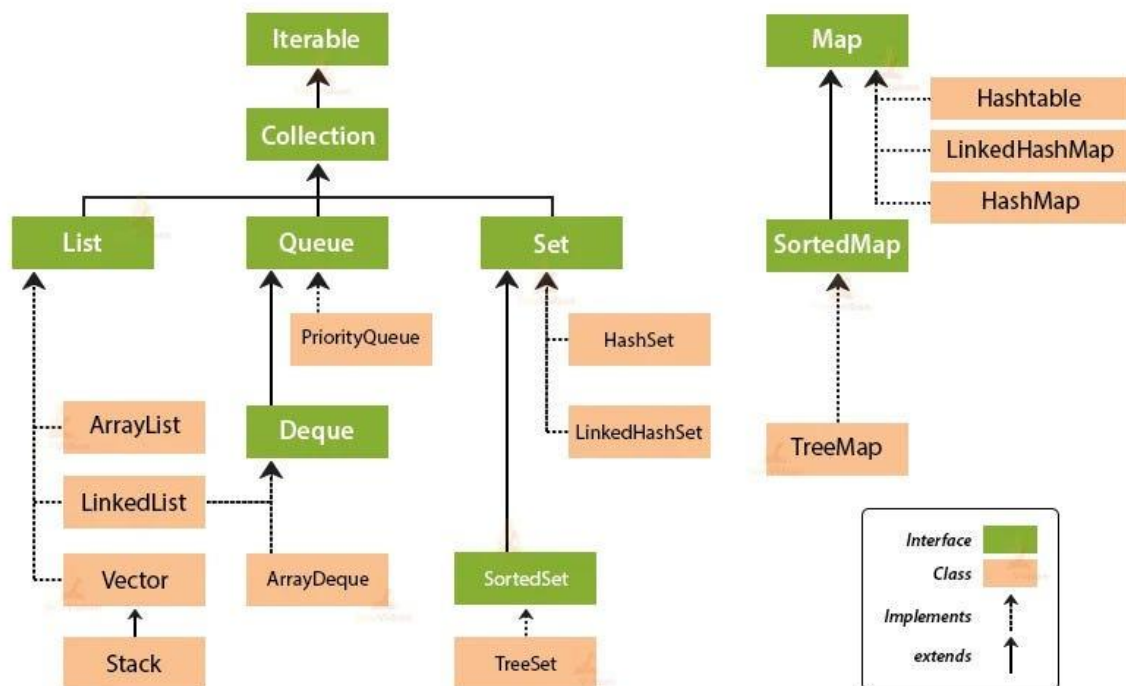
Java Collection Framework is a unified architecture of **interfaces and classes** that provides **ready-made data structures** for storing and manipulating groups of objects.

Interface tells WHAT type of collection it is.

Class tells HOW that collection is implemented.

Interface	Purpose
Iterable	Allows objects to be traversed using for-each
Collection	Represents a group of objects
List	Ordered collection, duplicates allowed
Set	No duplicate elements
Queue	Elements waiting to be processed

Collection Framework Hierarchy in Java



Iterable

Iterable means the collection can be traversed one by one, usually using a for-each loop.

```
ArrayList<String> names = new ArrayList<>();
names.add("Ravi");
names.add("Kumar");
names.add("Ravi");
```

```
for (String n : names) {  
    System.out.println(n);  
}
```

Ravi

Kumar

Ravi

```
for (String n : names) {  
    System.out.println(n);  
}
```

This will internally converted as

```
Iterator<String> it = names.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

Here, ArrayList implements List, List extends Collection, and Collection extends Iterable

ArrayList

An **ArrayList in Java** is a resizable-array implementation of the List interface found in the [java.util package](#).

Key Characteristics

- Dynamic Sizing
- Ordered Collection
- Allows Duplicates
- Objects Only
- Not Thread-Safe

Iterator

Iterator is an interface in java.util used to traverse elements of a collection one by one.

Method	Purpose
hasNext()	Checks whether another element exists
next()	Returns the next element
remove()	Removes the current element
forEachRemaining()	Processes all remaining elements using the given action

```
import java.util.*;
```

```
public class IteratorExample {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>();  
        list.add("Ravi");  
        list.add("Anu");  
        list.add("Kumar");  
        System.out.println("Before removing:");  
        Iterator<String> it = list.iterator();  
        while (it.hasNext()) {  
            String name = it.next();  
            System.out.println(name);  
        }  
        it = list.iterator();  
        while (it.hasNext()) {  
            String name = it.next();  
            if (name.equals("Anu")) {  
                it.remove();  
            }  
        }  
        System.out.println("\nAfter removing Anu:");  
        it = list.iterator();  
        while (it.hasNext()) {  
            String name = it.next();  
            System.out.println(name);  
        }  
    }  
}
```

Before removing:

Ravi

Anu

Kumar

After removing Anu:

Ravi

Kumar

```
import java.util.*;

public class ForEachRemainingExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Ravi");
        list.add("Anu");
        list.add("Kumar");
        list.add("Priya");
        Iterator<String> it = list.iterator();
        // Read the first element
        System.out.println("First element: " + it.next());
        // Process all remaining elements
        System.out.println("Remaining elements:");
        it.forEachRemaining(name -> {
            System.out.println(name);
        });
    }
}
```

First element: Ravi

Remaining elements:

Anu

Kumar

Priya

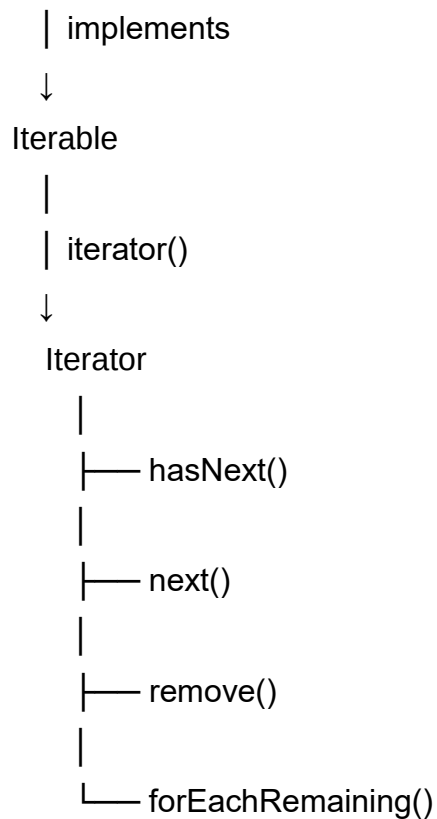
it.forEachRemaining(name -> {System.out.println(name);});

This lambda expression will be internally converted as

```
while (it.hasNext()) {
    String name = it.next();
    System.out.println(name);
}
```

ArrayList

|



Iterable	Iterator
Represents a collection that can be iterated	Represents the mechanism for iterating
Provides iterator()	Provides hasNext(), next() etc.
Usually used with enhanced for loop	Used for explicit traversal
Gives an Iterator	Traverses elements

Collection

Collection represents a group of objects. Collection is an interface. We normally create an object of a class such as ArrayList

```

import java.util.*;
class Example {
public static void main(String[] args) {
  Collection<String> c = new ArrayList<>();
  // ArrayList implements the Collection interface
  c.add("A");
  c.add("B");
  c.add("C");
  System.out.println(c);
}
  
```

```

}
[A, B, C]
Collection<String> c = new LinkedList<>();
c.add("A");
c.add("B");
c.add("C");
System.out.println(c);
[A, B, C]

```

List

A List is another interface:

Maintains order

Allows duplicates

Allows index access

```

List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Apple");
System.out.println(list);
System.out.println(list.get(1));
[Apple, Banana, Apple]
Banana

```

Collection vs. List

Operation	Collection	List	Example
Add element	☐	☐	add("Ravi")
Remove element	☐	☐	remove("Ravi")
Check element	☐	☐	contains("Ravi")
Find size	☐	☐	size()
Check empty	☐	☐	isEmpty()
Clear all elements	☐	☐	clear()
Convert to array	☐	☐	toArray()
Iterate	☐	☐	iterator()
Access by index	☐	☐	get(2)
Change by index	☐	☐	set(2, "Kumar")
Insert at specific index	☐	☐	add(2, "Kumar")
Remove by index	☐	☐	remove(2)

Find first index	☐	☐	indexOf("Ravi")
Find last index	☐	☐	lastIndexOf("Ravi")
Get ListIterator	☐	☐	listIterator()
Get sub-list	☐	☐	subList(1, 4)

ArrayList

```
ArrayList<String> a = new ArrayList<>();
a.add("A");
a.add("B");
a.add("C");
System.out.println(a);
[A, B, C]
```

LinkedList

LinkedList implements List and Deque.

```
LinkedList<String> l = new LinkedList<>();
l.add("A");
l.add("B");
l.add("C");
System.out.println(l);
[A, B, C]
```

Vector

```
Vector<Integer> v = new Vector<>();
v.add(10);
v.add(20);
v.add(30);
System.out.println(v);
[10, 20, 30]
```

Vector is synchronized by default whereas ArrayList is not synchronized by default. Vector performance is generally slower than ArrayList due to synchronization.

Stack

```
Stack<Integer> s = new Stack<>();
```

```
s.push(10);  
s.push(20);  
s.push(30);  
System.out.println(s.pop());
```

30

Queue

```
Queue<String> q = new LinkedList<>();  
q.add("A");  
q.add("B");  
q.add("C");  
System.out.println(q.remove());
```

A

PriorityQueue

A PriorityQueue removes elements according to their priority, rather than simply insertion order.

```
PriorityQueue<Integer> q = new PriorityQueue<>();  
q.add(30);  
q.add(10);  
q.add(20);  
System.out.println(q.poll());
```

10

Smallest number = highest priority

poll() = remove the front/highest-priority element

Deque (Double Ended Queue)

```
Deque<Integer> d = new ArrayDeque<>();  
d.addFirst(10);  
d.addLast(20);  
System.out.println(d.removeFirst());  
System.out.println(d.removeLast());
```

10

ArrayDeque

ArrayDeque is a class that implements the Deque interface

```
ArrayDeque<Integer> d = new ArrayDeque<>();  
d.addFirst(10);  
d.addLast(20);  
d.addLast(30);  
System.out.println(d);  
[10, 20, 30]
```

Set

A Set does not allow duplicate elements.

```
Set<String> s = new HashSet<>();  
s.add("Apple");  
s.add("Banana");  
s.add("Apple");  
System.out.println(s);  
[Apple, Banana]
```

HashSet

HashSet is a commonly used implementation of Set.

```
HashSet<Integer> s = new HashSet<>();  
s.add(30);  
s.add(10);  
s.add(20);  
s.add(10);  
System.out.println(s);  
[20, 10, 30]
```

LinkedHashSet

LinkedHashSet also does not allow duplicates, **but it maintains insertion order.**

```
LinkedHashSet<Integer> s = new LinkedHashSet<>();
```

```
s.add(30);  
s.add(10);  
s.add(20);  
s.add(10);  
System.out.println(s);  
[30, 10, 20]
```

HashSet → unique, order not guaranteed

LinkedHashSet → unique, insertion order maintained

SortedSet

SortedSet is an interface for a set whose elements are maintained in sorted order.

```
SortedSet<Integer> s = new TreeSet<>();  
s.add(30);  
s.add(10);  
s.add(20);  
System.out.println(s);  
[10, 20, 30]
```

TreeSet

TreeSet implements SortedSet

```
TreeSet<Integer> s = new TreeSet<>();  
s.add(50);  
s.add(20);  
s.add(40);  
System.out.println(s);  
[20, 40, 50]
```

Map

```
Map<Integer, String> m = new HashMap<>();  
m.put(101, "Ravi");  
m.put(102, "Kumar");  
System.out.println(m);
```

101 → Ravi

102 → Kumar

```
m.put(101, "Ravi");    // Add
m.get(101);           // Get value
m.remove(101);        // Remove
m.containsKey(101);   // Check key
m.containsValue("Ravi"); // Check value
m.size();             // Number of entries
m.clear();            // Remove all entries
```

HashMap

HashMap implements Map

```
HashMap<Integer, String> m = new HashMap<>();
m.put(101, "Ravi");
m.put(102, "Kumar");
m.put(103, "Arun");
System.out.println(m.get(102));
Kumar
```

```
HashMap<Integer, String> h = new HashMap<>();
h.put(10, "A");
h.put(20, "B");
h.put(null, "C"); // ✓ null key
h.put(30, null); // ✓ null value
h.put(40, null); // ✓ another null value
System.out.println(h);
{null=C, 20=B, 40=null, 10=A, 30=null}
```

LinkedHashMap

LinkedHashMap maintains the insertion order of entries.

```
LinkedHashMap<Integer, String> m = new LinkedHashMap<>();
m.put(103, "Arun");
m.put(101, "Ravi");
m.put(102, "Kumar");
```

```
System.out.println(m);  
{103=Arun, 101=Ravi, 102=Kumar}  
LinkedHashMap also allows null entries
```

HashMap = key-value pairs without guaranteed order

LinkedHashMap = key-value pairs with insertion order maintained

Hashtable

Hashtable is an older synchronized implementation of Map.

```
Hashtable<Integer, String> h = new Hashtable<>();  
h.put(30, "C");  
h.put(10, "A");  
h.put(20, "B");  
System.out.println(h);  
// Hashtable does NOT allow null key or null value  
// h.put(null, "X"); // ❌ Error  
// h.put(40, null); // ❌ Error  
{30=C, 20=B, 10=A}
```

HashMap vs. Hashtable

HashMap



Hashing

- + null allowed
- + not synchronized by default
- + generally faster

Hashtable



Hashing

- + synchronized methods
- + null NOT allowed
- + legacy

SortedMap

SortedMap maintains its keys in sorted order.

```
SortedMap<Integer, String> m = new TreeMap<>();
```

```
m.put(103, "Arun");
```

```
m.put(101, "Ravi");
```

```
m.put(102, "Kumar");
```

```
System.out.println(m);
```

```
{101=Ravi, 102=Kumar, 103=Arun}
```

Notice that the keys are sorted.

TreeMap

TreeMap implements SortedMap.

```
TreeMap<Integer, String> m = new TreeMap<>();
```

```
m.put(30, "C");
```

```
m.put(10, "A");
```

```
m.put(20, "B");
```

```
System.out.println(m);
```

```
{10=A, 20=B, 30=C}
```

Feature	HashMap	LinkedHashMap	Hashtable	TreeMap
Type	Class	Class	Class	Class
Stores	Key-Value	Key-Value	Key-Value	Key-Value
Order	☐ No guaranteed order	☐ Insertion order	☐ No guaranteed order	☐ Sorted by key
Duplicate keys	☐ No	☐ No	☐ No	☐ No
Duplicate values	☐ Yes	☐ Yes	☐ Yes	☐ Yes
null key	☐ One	☐ One	☐ No	☐ No
null values	☐ Yes	☐ Yes	☐ No	☐ Yes
Thread-safe	☐ No	☐ No	☐ Yes	☐ No
Main use	General-purpose map	Map + insertion order	Legacy synchronized map	Sorted key-value data

Type	Main idea	Example
List	Ordered + duplicates	ArrayList
Set	No duplicates	HashSet
SortedSet	No duplicates + sorted	TreeSet
Queue	Processing order	LinkedList
PriorityQueue	Priority-based removal	PriorityQueue
Deque	Both ends	ArrayDeque
Map	Key → Value	HashMap
SortedMap	Keys sorted	TreeMap
LinkedHashMap	Insertion order	LinkedHashMap
Hashtable	Older synchronized Map	Hashtable
Stack	LIFO	Stack